MLDS2 · From Notebook to Production · Part 1 of 3

From Notebook to Reproducible ML

Turning a notebook that worked once into code a team, a test suite and a scheduler can run

23 minutes Lecture: 16 November 2026

1 Train 2 Track 3 Serve 4 Monitor

The notebook that worked once

Four real cells from 01_notebook/electricity_forecasting.ipynb.

```
raw.loc[500:515, 'load_kw'] *= 1.2
raw.to_csv('electricity.csv', index=False)
```

Running it twice changes the data

The cell overwrites its own input file.

```
scaled = saved['scaler'].transform(new_features)
check = saved['model'].predict(scaled)
```

Every consumer repeats preprocessing

The model was saved apart from its fitted scaler.

```
RandomForestRegressor(n_estimators=40,
max_depth=10, ...)
train_test_split(X_scaled, y, test_size=0.2)
```

Settings and randomness live in cells

No config file, no seed: a rerun gives different numbers.

```
with open('model_final.pkl', 'wb') as f:
    pickle.dump({'model': model, ...}, f)
```

Results cannot be traced

'final' is the Ridge model, overwritten on every run.

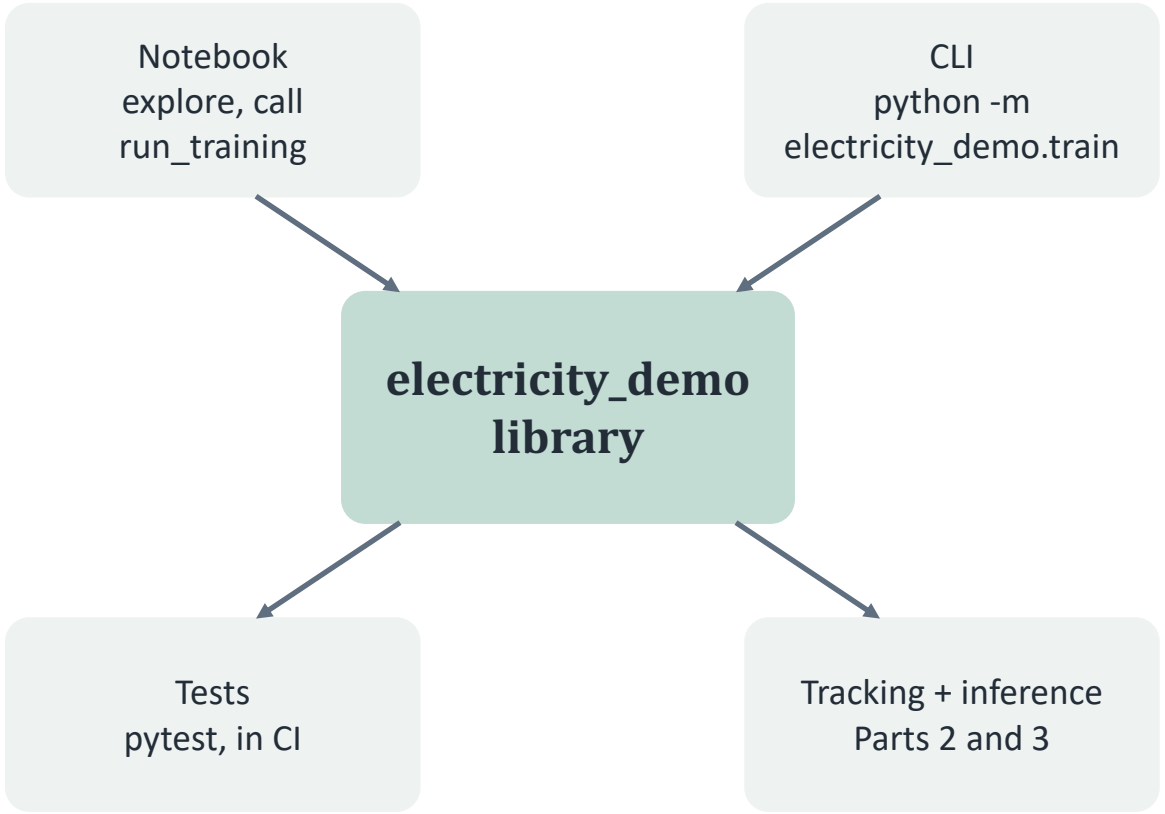
None of these is a modelling mistake a better metric would catch. They are software problems.

Library first: notebooks and scripts are thin clients

Move logic out of cells into a tested package; everything else calls it.

PROJECT LAYOUT (REAL)

```
02_reproducible_project/
├─ src/electricity_demo/    # the library
│    prepare · features · split
│    pipeline · evaluate · train
├─ configs/          ridge.yaml, random_forest.yaml
├─ tests/            pytest on synthetic data
├─ notebooks/        exploration.ipynb
├─ scripts/          one-off orchestration
├─ data/             DVC pointers, no CSVs in Git
└─ pyproject.toml · uv.lock · .python-version
```



If a notebook cell would be copied, it belongs in the library.

Keep fitted preprocessing with the model

Save one fitted object that turns features into predictions.

BEFORE: NOTEBOOK BUNDLE

```
pickle.dump({'model': model, 'scaler': scaler,
             'columns': cols}, f)

# every consumer must remember:
X = new[saved['columns']]
scaled = saved['scaler'].transform(X)
saved['model'].predict(scaled)
```

AFTER: SRC/ELECTRICITY_DEMO/PIPELINE.PY

```
Pipeline([
  ('preprocess', ColumnTransformer([
     ('numeric', StandardScaler(), lag_cols),
     ('calendar', OneHotEncoder(...),
                  ['hour', 'weekday'])])),
  ('model', RandomForestRegressor(...)),
])

pipeline.predict(X_new)    # the only call
```

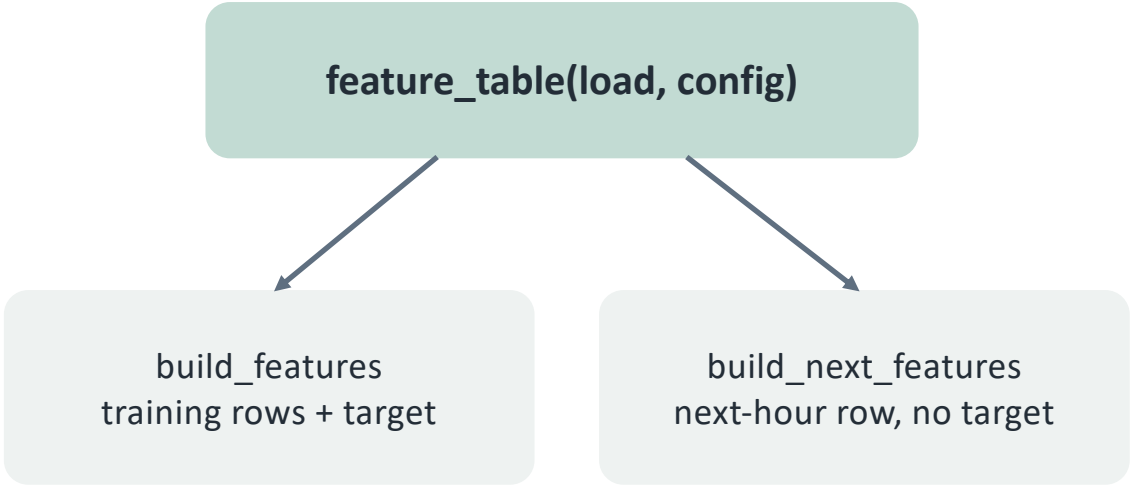
Fitted inside each CV fold, saved as one object (model.pkl), logged as one MLflow model in Part 2.

One feature definition for training and inference

Two implementations of the same feature drift apart silently.

SRC/ELECTRICITY_DEMO/FEATURES.PY

```
def feature_table(load, config):
    f = pd.DataFrame(index=load.index)
    for lag in config['lags']:
        f[f'lag_{lag}h'] = load.shift(lag)
    for w in config['rolling_windows']:
        f[f'mean_{w}h'] = (load.shift(1)
                           .rolling(w).mean())
    f['hour'] = f.index.hour
    f['weekday'] = f.index.dayofweek
    return f
```



`test_next_hour_features_match_training_without_the_target`

Inference (Part 3) calls build_next_features; it never re-implements a feature.

Training-serving skew starts with copied feature code. Share one function, and test that both paths agree.

Dependencies: consumer vs training

Declare them in pyproject.toml, lock them in uv.lock, install them the same way everywhere.

PYPROJECT.TOML (ABRIDGED)

```
[project]
dependencies = [pandas, numpy,
                scikit-learn, pyyaml]

[project.optional-dependencies]
tracking = [mlflow]          # registry, inference
tuning   = [mlflow, optuna]

[dependency-groups]
train = [matplotlib, rich, dvc]  # training only
dev   = [pytest, ...]
```

`uv lock` — Exact versions for every package, committed as uv.lock

`uv sync --locked` — Installs exactly the lock; fails if it is out of date

`uv run --locked …` — Every command runs in that same environment

A lockfile does not pin the data, uncommitted code, OS and numerical libraries, or unseeded randomness.

A consumer installs the package without training tools; a developer gets everything with uv sync.

Configuration instead of hard-coded values

Change an experiment by changing a file, not by editing code.

CONFIGS/RANDOM_FOREST.YAML (ABRIDGED)

```
seed: 42
dataset: data/prepared/hourly.csv
features:
  lags: [1, 2, …, 24, 168]
  rolling_windows: [24, 168]
evaluation:
  horizon_hours: 1
  cv_splits: 3
  cv_validation_hours: 336
model:
  type: random_forest
  params: {n_estimators: 40, max_depth: 10}
```

1 **Validated when loaded**

config.load_training_config rejects unknown models, unsupported horizons and malformed search grids before any training starts.

2 **Saved with every run**

The resolved config.yaml and its hash go into the run folder and manifest.

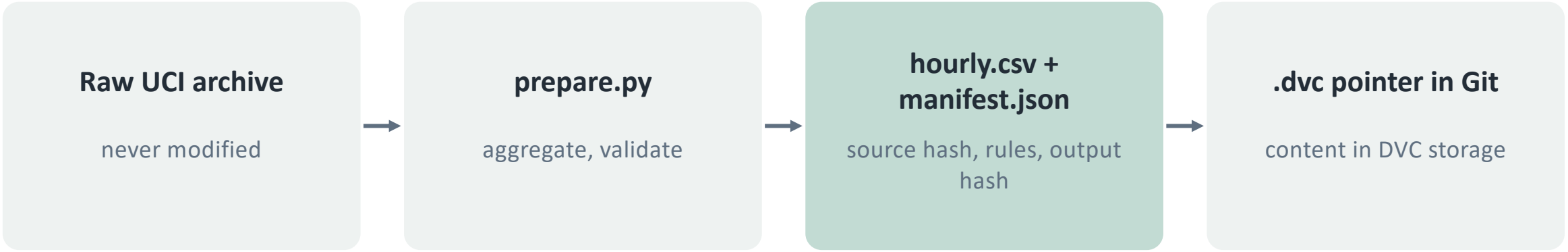
3 **One place per decision**

Features, evaluation design and model settings are no longer scattered through cells.

Configs are code: they are versioned in Git and reviewed like code.

Data identity: which data trained this model?

Raw data stays untouched; every derived file is identified by its hash.



Training refuses data that does not match its manifest. Tested by test_unrecorded_data_change_is_rejected.

Git versions the pointer, DVC stores the bytes. The run manifest records the hash, so a result can name its exact data.

Testing ML code: contracts, not accuracy

Tests check behaviour on small synthetic data. They run on every change.

Checks	Example test in our repository
Data contract	`test_invalid_hourly_data_is_rejected`
No future information	`test_current_and_future_targets_cannot_change_current_or_past_features`
Preprocessing fitted in-fold	`test_scaler_is_fitted_only_on_fold_training_rows`
Train / inference parity	`test_next_hour_features_match_training_without_the_target`
Save and reload	`test_training_repeats_and_saved_pipeline_reproduces_predictions`
Invalid input rejected	`test_next_hour_features_reject_short_or_gapped_history`
Failures are visible	`test_training_failure_is_recorded`

TESTS.YML (GITHUB ACTIONS)

```
on: [push, pull_request]
steps:
- uses: astral-sh/setup-uv@v6
- run: uv sync --locked
       --extra tuning
- run: uv run --locked
       --extra tuning
       python -m pytest -q
```

Tests are not quality gates. "Is the model good enough to ship?" is a separate, explicit check (Part 2).

32 tests, about 40 seconds, no dataset download.

One headless command

The same function runs from a terminal, a notebook, CI or a scheduler.

```
uv run --locked python -m electricity_demo.train --config configs/random_forest.yaml
```

INPUTS

config file
data version (hash)
uv.lock

EXECUTION

```
load_development
fit_and_evaluate
save_run
```

OUTPUTS: OUTPUTS/<RUN-ID>/

config.yaml · metrics.json
predictions.csv · model.pkl
feature_spec.json · manifest.json

manifest.json: Git commit and dirty flag, data hash, lockfile hash, resolved config and split boundaries.

No cell order, no manual steps: if it runs once, it runs again.

A week later: seven run folders

We have reproducible runs. We do not yet have a way to manage them.

```
2026-11-12_17-40-17Z_ridge_v1_760c2f/
2026-11-12_17-42-27Z_random_forest_v1_1ea427/
2026-11-12_18-19-54Z_ridge_v1_e7f336/
2026-11-12_18-21-13Z_random_forest_v1_cf4f24/
…  and comparison.md, written by our own report.py
```

This works for one person on one laptop. How does a team compare hundreds of runs, and find the exact model an application should use?

Part 2: experiment tracking and model management

Backup: the evaluation protocol in code

Covered by Part 1's code and tests; not part of the timed flow.

Chronological folds	Three expanding windows of 336 hours; no shuffling (split.py) `test_folds_respect_prediction_origins_and_exclude_final_test`
Final test opt-in	Scored only with --final-test, after selection `test_default_training_does_not_score_final_test`
Supported scope	One-hour horizon only; other horizons are rejected `test_unsupported_forecast_horizon_is_rejected`
Repeatability	Same config, data and lock: predictions equal within 1e-9 `test_training_repeats_and_saved_pipeline_reproduces_predictions`